UNIT-4
LOADERS AND LINKERS

The Source Program written in assembly language or high level language will be converted to object program, which is in the machine language form for execution. This conversion either from assembler or from compiler, contains translated instructions and data values from the source program, or specifies addresses in primary memory where these items are to be loaded for execution.
This contains the following three processes, and they are,

- **Loading** - which allocates memory location and brings the object program into memory for execution - (Loader)
- **Linking**- which combines two or more separate object programs and supplies the information needed to allow references between them - (Linker)
- **Relocation** - which modifies the object program so that it can be loaded at an address different from the location originally specified - (Linking Loader)

Basic Loader Functions:

- A **Loader** brings an object program into memory and starting its execution.
- A **Linker** performs the linking operations and a separate loader to handle relocation and loading.

The role of loader is as shown in below figure here translator may be assembler/complier, which generates the object program and later loaded to the memory by the loader for execution.

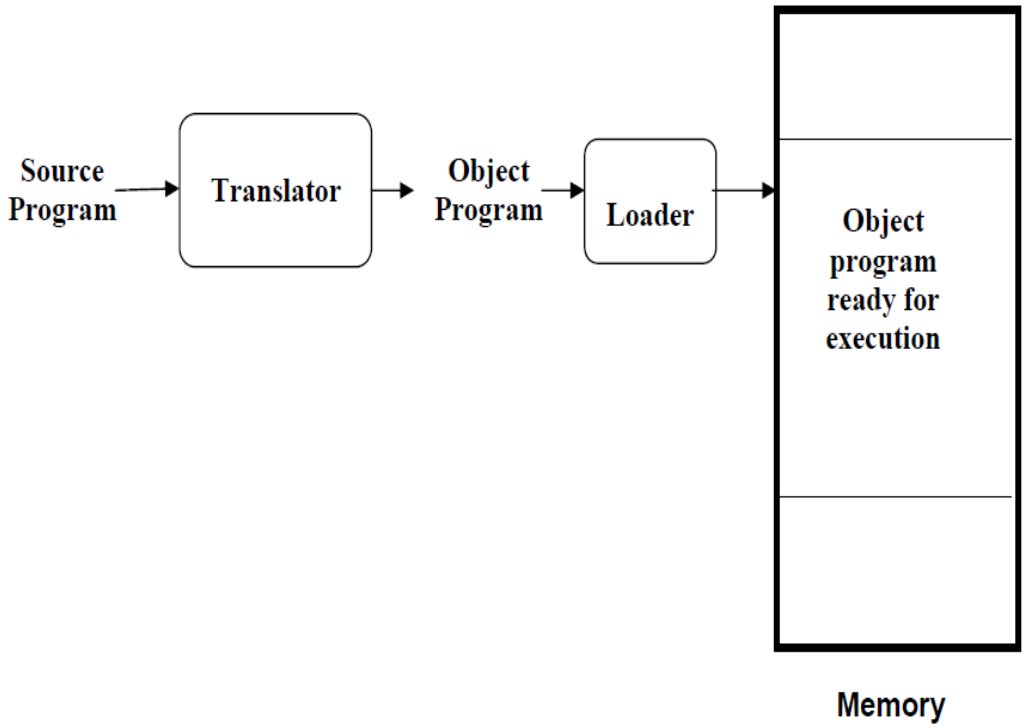


Fig: ROLE OF LOADER

In below figure translator is specifically an assembler, which generates the object loaded, which becomes input to the loader.

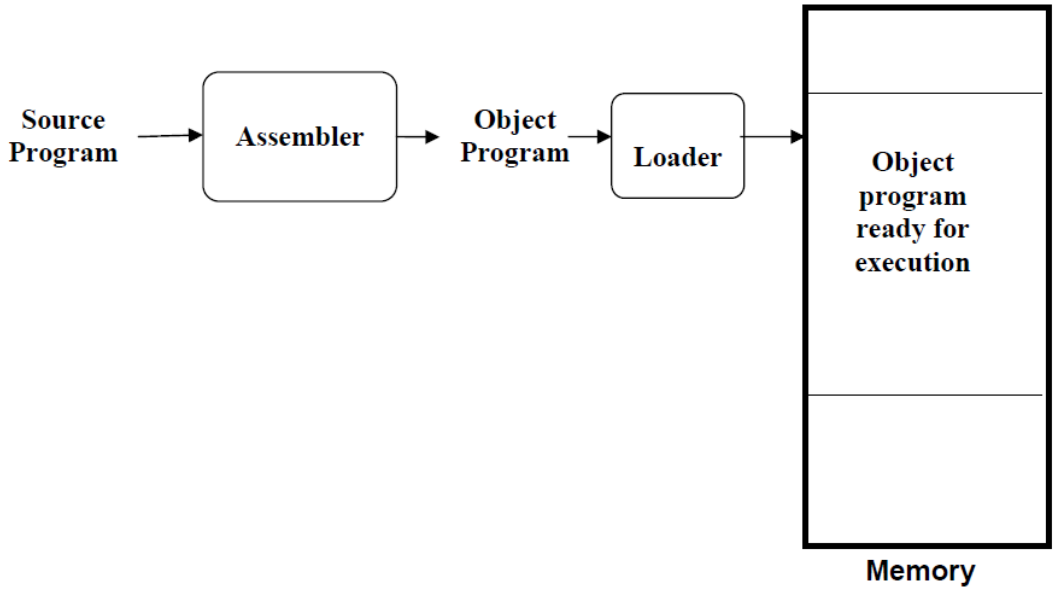


Fig: THE ROLE OF LOADER WITH ASSEMBLER

The below figure shows the role of both loader and linker.

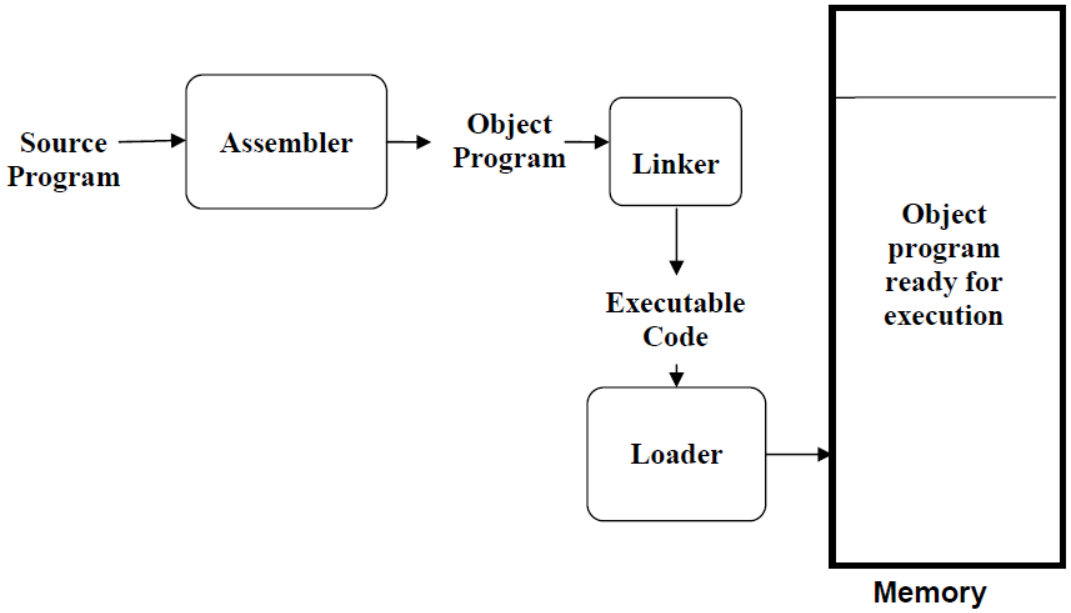


Fig: THE ROLE OF BOTH LOADER AND LINKER

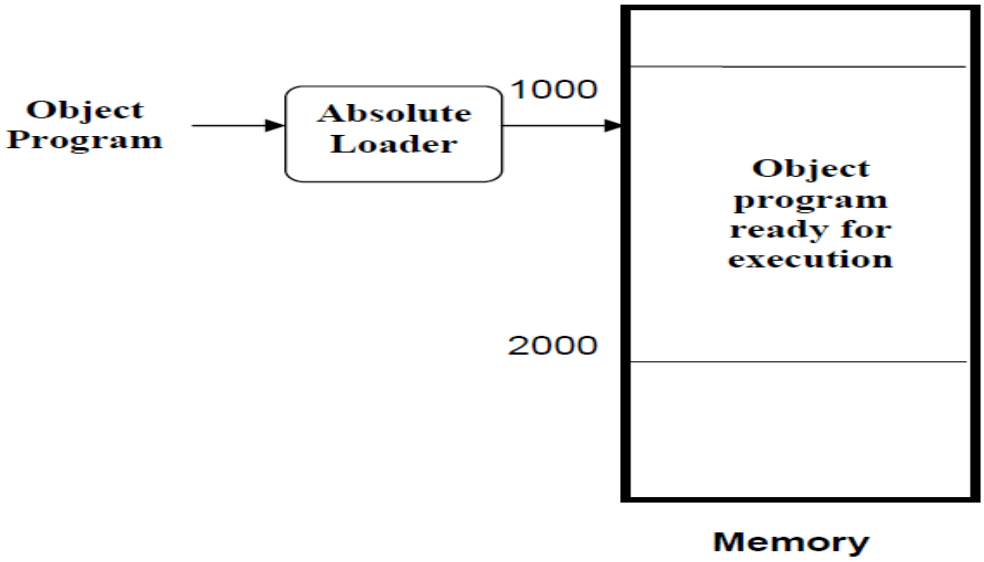
Type of Loaders:

The different types of loaders are, absolute loader, bootstrap loader, relocating loader (relative loader), and, direct linking loader. The following sections discuss the functions and design of all these types of loaders.

Design of an Absolute Loader:

- The operation of absolute loader is very simple.
- The object code is loaded to specified locations in the memory.
- At the end the loader jumps to the specified address to begin execution of the loaded program.

The role of Absolute loader is shown in below figure.



- The algorithm for this type of loader is given here. The object code and, the object program loaded into memory by the absolute loader are also shown.
- Each byte of assembled code is given using its hexadecimal representation in character form.
- Easy to read by human beings. Each byte of object code is stored as a single byte.
- Most machine store object programs in a binary form, and we must be sure that our file and device conventions do not cause some of the program bytes to be interpreted as control characters.

```
HCOPY  001000001007A
T001000 1E 14 10 33 48 20 39 00 10 36 28 10 30 30 10 15 48 20 61 3C 10 03 00 10 2A 0C 10 39 00 10 2D
T001010 1E 15 0C 10 36 48 20 61 08 10 33 4C 00 00 45 4F 46 00 00 03 00 00 00
T002039 1E 04 10 30 00 10 30 E0 20 5D 30 20 3F D8 20 5D 28 10 30 30 20 57 54 90 39 2C 20 5E 38 20 3F
T002057 1C 10 10 36 4C 00 00 0F 10 01 00 00 04 10 30 E0 20 79 30 20 64 50 90 39 DC 20 79 2C 10 36
T002073 07 38 20 64 4C 00 00 05
E001000
```

(a) Object program

```
begin
    read Header record
    verify program name and length
    read first Text record
    while record type ≠ 'E' do
      begin
        {if object code is in character form, convert into
           internal representation}
        move object code to specified location in memory
        read next object program record
      end
    jump to address specified in End record
end
```

"14" occupies two bytes if it is represented in char form.

When loaded into memory, "14" should occupy only one byte.

Figure 3.2 Algorithm for an absolute loader.

Memory address	Contents			
0000	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
0010	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮
0FF0	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
1000	14103348	20390010	36281030	30101548
1010	20613C10	0300102A	0C103900	102D0C10
1020	36482061	0810334C	0000454F	46000003
1030	000000xx	xxxxxxxx	xxxxxxxx	xxxxxxxx ← COPY
⋮	⋮	⋮	⋮	⋮
2030	xxxxxxxx	xxxxxxxx	xx041030	001030E0
2040	205D3020	3FD8205D	28103030	20575490
2050	392C205E	38203F10	10364C00	00F10010
2060	00041030	E0207930	20645090	39DC2079
2070	2C103638	20644C00	0005xxxx	xxxxxxxx
2080	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮

(b) Program loaded in memor

No text record corresponds here. XXX indicates that the previous contents of these locations remain unchanged.

Advantages:

- It is simple to implement
- This scheme allows multiple programs or the source programs written different languages. If there are multiple programs written in different languages then the respective language assembler will convert it to the language and a common object file can be prepared with all the resolution.
- The task of loader becomes simpler as it simply obeys the instruction regarding where to place the object code in the main memory.
- The process of execution is efficient.

Disadvantages:

- A portion of memory is wasted because the memory occupied by the assembler is unavailable to the object program.
- It is necessary to re-translate (assemble) the user's program file every time it is run.
- It is very difficult to handle multiple segments, especially if the source programs are in different.
- Actual load address must be specified
- The programmer must be careful not to assign two subroutines to the same or overlapping locations

A Simple Bootstrap Loader:

- When a computer is first turned on or restarted, a special type of absolute loader, called bootstrap loader is executed.
- This bootstrap loads the first program to be run by the computer -- usually an operating system.
- The bootstrap itself begins at address 0. It loads the OS starting address 0x80. No Header record, End record or control information, the object code is consecutive bytes of memory.

The algorithm for the bootstrap loader is as follows

Begin
X=0x80 (the address of the next memory location to be loaded
Loop
A⟶GETC (and convert it from the ASCII character
code to the value of the hexadecimal digit)
save the value in the high-order 4 bits of S
A⟶ GETC
combine the value to form one byte A ----- ->(A+S)
store the value (in A) to the address in register X
X------>X+1
End
It uses a subroutine GETC, which is
GETC A----->read one character
if A=0x04 then jump to 0x80
if A<48 then GETC
A ---> A- 48 (0x30)
if A<10 then return
A---->A-7
return

The Source Code for BOOTSTRAP LOADER is given below

```
BOOT    START   0       BOOTSTRAP LOADER FOR SIC/XE
.
. THIS BOOTSTRAP READS OBJECT CODE FROM DEVICE F1 AND ENTERS IT
. INTO MEMORY STARTING AT ADDRESS 80 (HEXADECIMAL). AFTER ALL OF
. THE CODE FROM DEVF1 HAS BEEN SEEN ENTERED INTO MEMORY, THE
. BOOTSTRAP EXECUTES A JUMP TO ADDRESS 80 TO BEGIN EXECUTION OF
. THE PROGRAM JUST LOADED.  REGISTER X CONTAINS THE NEXT ADDRESS
. TO BE LOADED.
.
        CLEAR   A       CLEAR REGISTER A TO ZERO
        LDX     #128    INITIALIZE REGISTER X TO HEX 80
LOOP    JSUB    GETC    READ HEX DIGIT FROM PROGRAM BEING LOADED
        RMO     A,S     SAVE IN REGISTER S
        SHIFTL  S,4     MOVE TO HIGH-ORDER 4 BITS OF BYTE
        JSUB    GETC    GET NEXT HEX DIGIT
        ADDR    S,A     COMBINE DIGITS TO FORM ONE BYTE
        STCH    0,X     STORE AT ADDRESS IN REGISTER X
        TIXR    X,X     ADD 1 TO MEMORY ADDRESS BEING LOADED
        J       LOOP    LOOP UNTIL END OF INPUT IS REACHED
.
. SUBROUTINE TO READ ONE CHARACTER FROM INPUT DEVICE AND
. CONVERT IT FROM ASCII CODE TO HEXADECIMAL DIGIT VALUE. THE
. CONVERTED DIGIT VALUE IS RETURNED IN REGISTER A. WHEN AN
. END-OF-FILE IS READ, CONTROL IS TRANSFERRED TO THE STARTING
. ADDRESS (HEX 80).
.
GETC    TD      INPUT   TEST INPUT DEVICE
        JEQ     GETC    LOOP UNTIL READY
        RD      INPUT   READ CHARACTER
        COMP    #4      IF CHARACTER IS HEX 04 (END OF FILE),
        JEQ     80          JUMP TO START OF PROGRAM JUST LOADED
        COMP    #48     COMPARE TO HEX 30 (CHARACTER '0')
        JLT     GETC    SKIP CHARACTERS LESS THAN '0'
        SUB     #48     SUBTRACT HEX 30 FROM ASCII CODE
        COMP    #10     IF RESULT IS LESS THAN 10, CONVERSION IS
        JLT     RETURN      COMPLETE. OTHERWISE, SUBTRACT 7 MORE
        SUB     #7          (FOR HEX DIGITS 'A' THROUGH 'F')
RETURN  RSUB            RETURN TO CALLER
INPUT   BYTE    X'F1'   CODE FOR INPUT DEVICE
        END     LOOP
```

Convert "14" in char form to "14" in byte form

Convert "1" in char form to "1" in its ASCII code

Working of Bootstrap Loader:

- Much of the work of the bootstrap loader is performed by the subroutine GETC. This subroutine reads one character from device F1 and converts it from the ASCII character code to the value of the hexadecimal digit that is represented by the character.
- For Example, The ASCII code for the character "0" (hex 30) is converted to numeric value 0. Likewise, the ASCII codes for "1" through "9" (hex 31 through 39) are converted to numeric values 1 through 9, and the codes for "A" through "F" (hex 41 through 46) are converted to the values 10 through

15. this accomplished by subtracting 48 (hex 30) from the character codes for "0" through "9", and subtracting 55 (hex 37) from the code for "A" through "F".

- The subroutine GETC jumps to address 80 when an end-of-file (hex 04) is read from device F1.
- The main loop of the bootstrap keeps the address of the next memory location to be loaded in register X. GETC is used to read and convert a pair of characters from device F1. These two hexadecimal digit values are combined into a single byte by shifting the first one left 4 bit positions and adding the second to it.
- The resulting byte is stored at the address currently in register X, using a STCH instruction that refers to location 0 using indexed addressing.
- The TIXR instruction is then used to add 1 to the value in register X.

Machine-Dependent Loader Features:

- Absolute loader is simple and efficient, but the scheme has potential disadvantages.
- One of the most disadvantage is the programmer has to specify the actual starting address, from where the program to be loaded.
- This does not create difficulty, if one program to run, but not for several programs. Further it is difficult to use subroutine libraries efficiently.
- This needs the design and implementation of a more complex loader. The loader must provide program relocation and linking, as well as simple loading functions.

Relocation:

- The concept of program relocation is, the execution of the object program using any part of the available and sufficient memory.
- The object program is loaded into memory wherever there is room for it. The actual starting address of the object program is not known until load time.
- Relocation provides the efficient sharing of the machine with larger memory and when several independent programs are to be run together. It also supports the use of subroutine libraries efficiently.
- Loaders that allow for program relocation are called relocating loaders or relative loaders.

Methods for specifying relocation:

1. Modification Record.
2. Relocation Bit.

<u>Modification Record:</u>

- A modification record is used to describe each part of the object code that must be changed when the program is relocated.
- The SIC/XE program is used to illustrate the modification record because of the only portions of the assembled program that contain actual addresses are the extended format instructions.
- Each modification record specifies the starting address and length of the field whose value is to altered. It then describes the modification to be performed.

- The modification record scheme is a convenient means for specifying program relocation.
- A modification record is used in complex machines and is also called Relocation and Linkage Directory (RLD) specification.

The format of the modification record (M) is as follows. The object program with relocation by Modification records is also shown here

Modification record
- col 1: M
- col 2-7: relocation address
- col 8-9: length (halfbyte)
- col 10: flag (+/-)
- col 11-17: segment name

```
HCOPY  000000001077
T0000001D17202D69202D4B10103603202629000033200 74B10105D3F2FEC032010
T00001D130F20160100030F200D4B10105D3E2003454F46
T001036 1DB410B400B44075101000E32019332FFADB2013A00433200857C003B850
T001053 1D3B2FEA1340004F0000F1B410774000E32011332FFA53C003DF2008B850
T00107007 3B2FEF4F000005
M00000705+COPY
M00001405+COPY
M00002705+COPY
E000000
```

Algorithm For SIC/XE Relocation Loader:

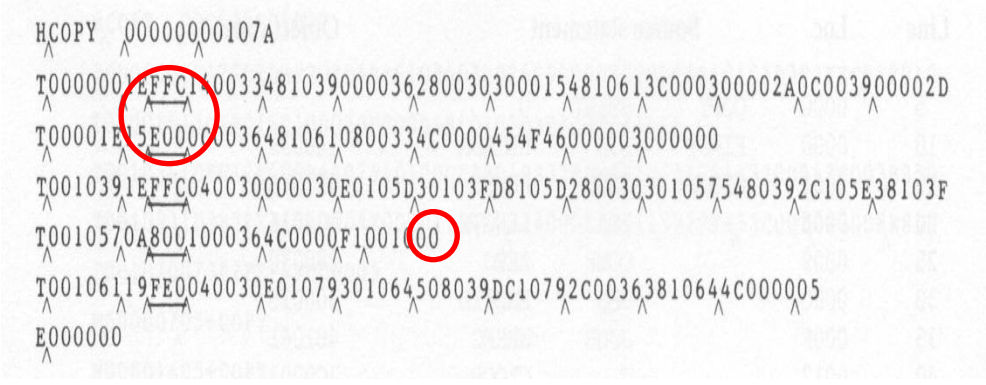
```
begin{SIC/XE}
    get PROGADDR from operating system
    while not end of input do
    begin
        read next record
        while record type != 'E' do
        begin
            read next input record
            while record type = 'T' then
            begin
                move object code from record to location
                ADDR + specified address
            end
            while record type = 'M'
                add PROGADDR at the location PROGADDR +
                 specified address
        end
    end
end {SIC/XE}
```

Relocation Bit:

- If an object needs too many modification records, it would be more efficient to use a relocation bit mask to indicate where in the object program should be modified when the object program is loaded.
- Relocation bit method is used for simple machines mainly SIC.
- Relocation bit is 0: no modification is necessary, and is 1: modification is needed. This is specified in the columns 10-12 of text record (T), the format of text record, along with relocation bits is as follows.

Text record
col 1: T
col 2-7: starting address
col 8-9: length (byte)
col 10-12: relocation bits
col 13-72: object code

- Twelve-bit mask is used in each Text record:
  - col:10-12 – relocation bits
  - since each text record contains less than 12 words
  - unused words are set to 0
  - any value that is to be modified during relocation must coincide with one of these 3-byte segments
- The bit mask FFC (representing the bit string 111111111100) in the first text record specifies that all 10 words of object code are to be modified during relocation.
- The mask E00 in the second text record specifies that the first three words to be modified. The remainder of the object code in this record represents data constants.
- The bit mask are underlined in object program.



This one-byte “F1” makes the LDX instruction on line 210 begins a new text record. This is because each relocation bit should be associated with a three-byte word. However, this data item occupies only one byte, which violates the Alignment rule.

SIC Relocation Loader Algorithm:

```
begin
    get PROGADDR from operating system
    while not end of input do
    begin
        read next text record
        while record type!= 'E' do
        while record type!= 'T'
        begin
            get length= second data
            mask bits(M) as third data
            for(i=0;i<length;i++)
            if Mi =1 then
            add PROGADDR at the location PROGADDR+ specified
            address
            else
            move object code from record to location PROGADDR
            +specified address
            Read next record
        end
    end
end
```

Program Linking:

- A program may be composed of many control sections.
- These control sections may be assembled separately.
- These control sections may be loaded at different addresses in memory.
- The Goal of program linking is to resolve the problems with external references (EXTREF) and external definitions (EXTDEF) from different control sections.

EXTDEF (external definition) - The EXTDEF statement in a control section names symbols , called external symbols, that are defined in this (present) control section and may be used by other sections.

eg: EXTDEF BUFFER, BUFFEND, LENGTH
EXTDEF LISTA, ENDA

EXTREF (external reference) - The EXTREF statement names symbols used in this (present) control section and are defined elsewhere.

eg: EXTREF RDREC, WRREC
EXTREF LISTB, ENDB, LISTC, ENDC

How to implement EXTDEF and EXTREF

- The assembler must include information in the object program that will cause the loader to insert proper values where they are required – in the form of Define record (D) and, Refer record(R).

Define record

The format of the Define record (D) along with examples is as shown here**.**

Col. 1 D

Col. 2-7 Name of external symbol defined in this control section

Col. 8-13 Relative address within this control section (hexadecimal)

Col.14-73 Repeat information in Col. 2-13 for other external symbols

Example records

D∧LISTA∧000040∧ENDA∧ 000054

Refer record

The format of the Refer record (R) along with examples is as shown here**.**

Col. 1 R

Col. 2-7 Name of external symbol referred to in this control section

Col. 8-73 Name of other external reference symbols

Example records

R ∧LISTB ∧ENDB∧LISTC∧ENDC

- Here are the three programs named as PROGA, PROGB and PROGC, which are separately assembled and each of which consists of a single control section.
- LISTA, ENDA in PROGA, LISTB, ENDB in PROGB and LISTC, ENDC in PROGC are external definitions in each of the control sections.
- Similarly LISTB, ENDB, LISTC, ENDC in PROGA, LISTA, ENDA, LISTC, ENDC in PROGB, and LISTA, ENDA, LISTB, ENDB in PROGC, are external references.
- These sample programs given here are used to illustrate linking and relocation. The following figures give the sample programs and their corresponding object programs. Observe the object programs, which contain D and R records along with other records.

```
Loc       Source statement                          Object code

0000  PROGA  START   0
             EXTDEF  LISTA,ENDA
             EXTREF  LISTB,ENDB,LISTC,ENDC
             .
             .
             .
0020  REF1   LDA     LISTA                         03201D
0023  REF2   +LDT    LISTB+4                       77100004
0027  REF3   LDX     #ENDA-LISTA                   050014
             .
             .
             .
0040  LISTA  EQU     *
             .
             .
0054  ENDA   EQU     *
0054  REF4   WORD    ENDA-LISTA+LISTC              000014
0057  REF5   WORD    ENDC-LISTC-10                 FFFFF6
005A  REF6   WORD    ENDC-LISTC+LISTA-1            00003F
005D  REF7   WORD    ENDA-LISTA-(ENDB-LISTB)       000014
0060  REF8   WORD    LISTB-LISTA                   FFFFC0
             END     REF1
```

Loc		Source statement		Object code
0000	PROGB	START	0	
		EXTDEF	LISTB,ENDB	
		EXTREF	LISTA,ENDA,LISTC,ENDC	
		.		
		.		
		.		
0036	REF1	+LDA	LISTA	03100000
003A	REF2	LDT	LISTB+4	772027
003D	REF3	+LDX	#ENDA-LISTA	05100000
		.		
		.		
		.		
0060	LISTB	EQU	*	
		.		
		.		
0070	ENDB	EQU	*	
0070	REF4	WORD	ENDA-LISTA+LISTC	000000
0073	REF5	WORD	ENDC-LISTC-10	FFFFF6
0076	REF6	WORD	ENDC-LISTC+LISTA-1	FFFFFF
0079	REF7	WORD	ENDA-LISTA-(ENDB-LISTB)	FFFFF0
007C	REF8	WORD	LISTB-LISTA	000060
		END		

Loc		Source statement		Object code
0000	PROGC	START	0	
		EXTDEF	LISTC,ENDC	
		EXTREF	LISTA,ENDA,LISTB,ENDB	
		.		
		.		
		.		
0018	REF1	+LDA	LISTA	03100000
001C	REF2	+LDT	LISTB+4	77100004
0020	REF3	+LDX	#ENDA-LISTA	05100000
		.		
		.		
		.		
0030	LISTC	EQU	*	
		.		
		.		
0042	ENDC	EQU	*	
0042	REF4	WORD	ENDA-LISTA+LISTC	000030
0045	REF5	WORD	ENDC-LISTC-10	000008
0048	REF6	WORD	ENDC-LISTC+LISTA-1	000011
004B	REF7	WORD	ENDA-LISTA-(ENDB-LISTB)	000000
004E	REF8	WORD	LISTB-LISTA	000000
		END		

Object Programs

```
HPROGA 000000000063
DLISTA 000040ENDA  000054
RLISTB ENDB  LISTC ENDC
:
:
T000020 0A 03201D 771000 04 050014
:
:
T000054 0F 000014 FFFFF6 000003 F000014 FFFFC0
M000024 05 +LISTB
M000054 06 +LISTC
M000057 06 +ENDC
M000057 06 -LISTC
M00005A 06 +ENDC
M00005A 06 -LISTC
M00005A 06 +PROGA
M00005D 06 -ENDB
M00005D 06 +LISTB
M000060 06 +LISTB
M000060 06 -PROGA
E000020
```

```
HPROGB 00000000007F
DLISTB 000060ENDB  000070
RLISTA ENDA  LISTC ENDC
:
:
T000036 0B 03100000 772027 05100000
:
:
T000070 0F 000000 FFFFF6 FFFFFF FFFFF0 000060
M000037 05 +LISTA
M00003E 05 +ENDA
M00003E 05 -LISTA
M000070 06 +ENDA
M000070 06 -LISTA
M000070 06 +LISTC
M000073 06 +ENDC
M000073 06 -LISTC
M000076 06 +ENDC
M000076 06 -LISTC
M000076 06 +LISTA
M000079 06 +ENDA
M000079 06 -LISTA
M00007C 06 +PROGB
M00007C 06 -LISTA
E
```

```
HPROGC 000000000051
DLISTC 000030ENDC  000042
RLISTA ENDA  LISTB ENDB
:
:
T000018 0C 03100000 771000 04 05100000
:
:
T000042 0F 000030 000008 000011 000000 000000
M000019 05 +LISTA
M00001D 05 +LISTB
M000021 05 +ENDA
M000021 05 -LISTA
M000042 06 +ENDA
M000042 06 -LISTA
M000042 06 +PROGC
M000048 06 +LISTA
M00004B 06 +ENDA
M00004B 06 -LISTA
M00004B 06 -ENDB
M00004B 06 +LISTB
M00004E 06 +LISTB
M00004E 06 -LISTA
E
```

Program Linking Example

- Notice that program A defines LISTA and ENDA, program B defines LISTB and ENDB, and program defines LISTC and ENDC.
- Notice that the definitions of REF1, REF2 ,….., to REF7 in all of these three control sections are the same.
- Therefore, after these three control sections are loaded, no matter where they are loaded, the values of REF1 to REF7 in all of these programs should be the same.

REF 1

- Program A
  - LISTA is defined in its own program and its address is immediately available. Therefore, we can simply use program counter-relative addressing
- Program B
  - Because LISTA is an external reference, its address is not available now. Therefore an extended-format instruction with address field set to 00000 is used. A modification record in inserted into the object code so that once LISTA's address is known, it can be added to this field.
- Program C
  - The same as that processed in Program B.

REF 2

- Program A
  - Because LISTB is an external reference, its address is not available now. Therefore an extended-format instruction with address field set to 00004 is used. A modification record is inserted into the object code so that once LISTB's address is available; it can be added to this field.
- Program B
  - LISTB is defined in its own program and its address is immediately available. Therefore, we can simply use program counter-relative addressing
- Program C
  - The same as that processed in Program A.

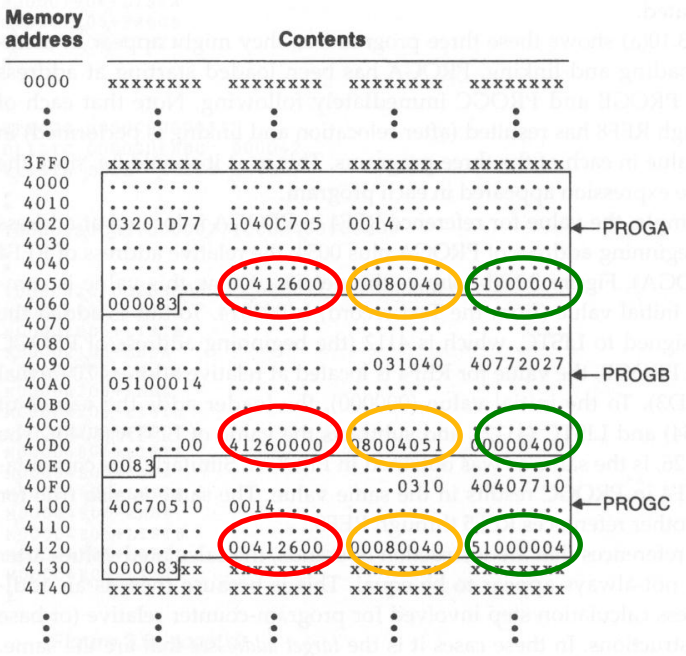
REF 3

- Program A
  - The difference between ENDA and LISTA (14) is immediately available during assembly.
- Program B
  - Because the values of ENDA and LISTA are unknown during assembly, we need to use an extended-format instruction with its address field set to 0.
  - Two modification records are inserted to the object program – one for +ENDA and the other for –LISTA.
- Program C
  - The same as that processed in Program B.

REF 4

- Program A
  - The difference between ENDA and LISTA can be known now. Only the value of LISTC is unknown. Therefore, an initial value of 000014 is stored with one modification record for LISTC.
- Program B
  - Because none of ENDA, LISTA, and LISTC's values can be known now, an initial value of 000000 is stored with three modification records for all of them.
- Program C
  - The value of LISTC is known now. However, the values for ENDA and LISTA are unknown. An initial value of 000030 is stored with two modification records for ENDA and LISTA.

After Loading into Memory:



Suppose that program A is loaded at 004000, program B at 004063, and program C at 0040E2.

Notice that REF4, REF5, REF6, and REF7 in all of these three programs have the same values.

REF 4 after Linking:

Load address for control sections

- PROGA 004000 63
- PROGB 004063 7F
- PROGC 0040E2 51

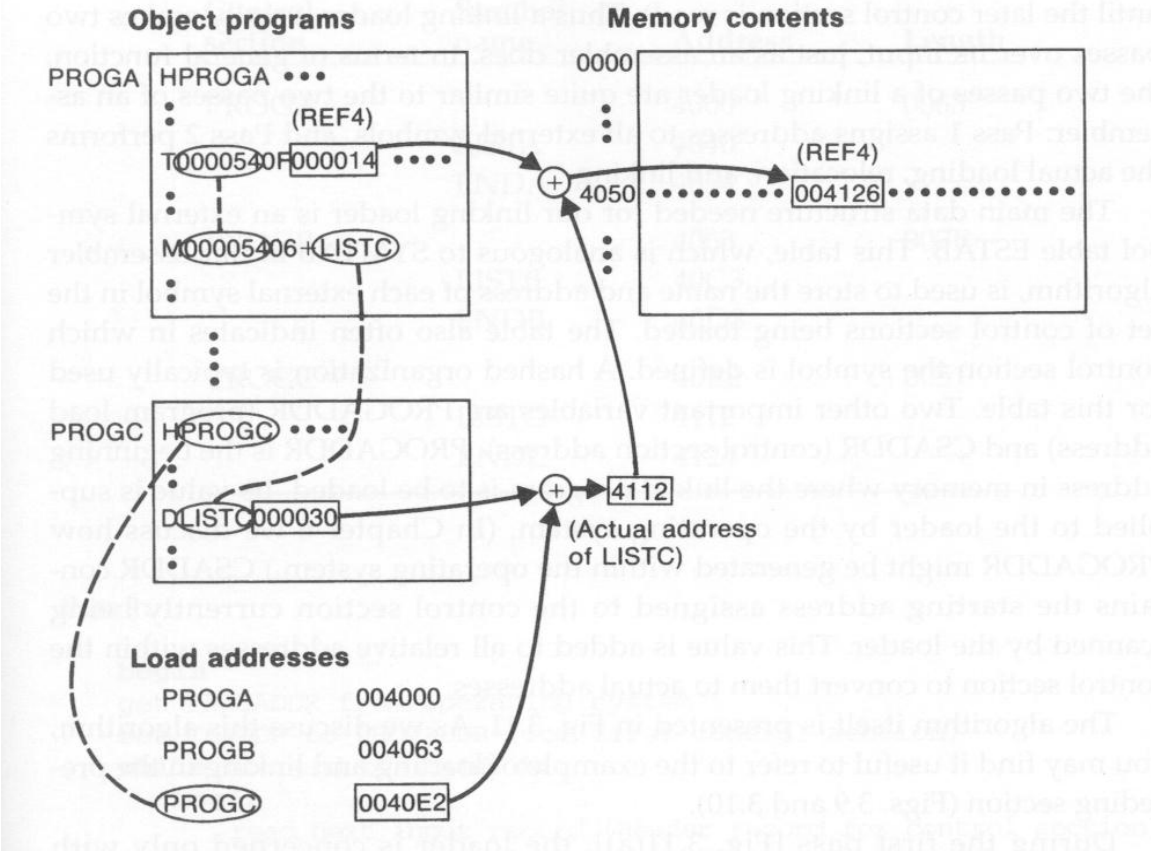
Load address for symbols

- LISTA: PROGA+0040=4040
- LISTB: PROGB+0060=40C3
- LISTC: PROGC+0030=4112

For Program A

- The address of REF4 is 4054 (4000 + 54) because program A is loaded at 4000 and the relative address of REF4 within program A is 54.
- The value of REF4 is 004126 because
  - The address of LISTC is 0040E2 (the loaded address of program C) + 000030 (the relative address of LISTC in program C)
  - 0040E2 + 000014 (constant already calculated) = 004126.

REF 4 in Program A
ENDA-LISTA+LISTC=14+4112=4126
(ENDA-LISTA = 14 (4054-4040)



Algorithm and Data Structures of Linking Loader:

- The algorithm for a linking loader is considerably more complicated than the absolute loader program, which is already given.
- The concept given in the program linking section is used for developing the algorithm for linking loader.
- The modification records are used for relocation so that the linking and relocation functions are performed using the same mechanism.
- Linking Loader uses two-pass logic.

Pass 1: Assign addresses to all external symbols
Pass 2: Perform the actual loading, relocation, and linking

ESTAB (external symbol table) is the main data structure for a linking loader.

ESTAB: This table, which is analogous to SYMTAB in our assembler algorithm, is used to store the name and address of each external symbol in the set of control sections being loaded. The ESTAB has four entries in it; they are name of the control section, the symbol appearing in the control section, its address and length of the control section. A hashed organization is typically used for this table.

Control section	Symbol name	Address	Length
PROGA		4000	0063
	LISTA	4040	
	ENDA	4054	
PROGB	4000+0063=	4063	007F
	LISTB	40C3	
	ENDB	40D3	
PROGC	4063+007F=	40E2	0051
	LISTC	4112	
	ENDC	4124	

Two other important variables are **PROGADDR** (program load address) and **CSADDR** (control section address).

PROGADDR: Is the beginning address in memory where the linked program is to be loaded. Its value is supplied to the loader by the OS.

CSADDR: Contains the starting address assigned to the control section currently being scanned by the loader. This value is added to all relative addresses within the control section to convert them to actual addresses.

Other Variables are:

CSLTH: Length of the control section.
EXECADDR: Execution Address.

Algorithm for Pass1 of a Linking Loader:

Pass 1:

```
begin
get PROGADDR from operating system
set CSADDR to PROGADDR {for first control section}
while not end of input do
   begin
      read next input record {Header record for control section}
      set CSLTH to control section length
      search ESTAB for control section name
      if found then
         set error flag {duplicate external symbol}
      else
         enter control section name into ESTAB with value CSADDR
      while record type ≠ 'E' do
         begin
            read next input record
            if record type = 'D' then
               for each symbol in the record do
                  begin
                     search ESTAB for symbol name
                     if found then
                        set error flag (duplicate external symbol)
                     else
                        enter symbol into ESTAB with value
                           (CSADDR + indicated address)
                  end {for}
         end {while ≠ 'E'}
      add CSLTH to CSADDR {starting address for next control section}
   end {while not EOF}
end {Pass 1}
```

Explanation for pass1 of Linking Loader:

- During Pass 1 the loader is concerned only with Header and Define record types in the control sections.
- The beginning load address for the linked program (PROGADDR) is obtained from the OS. This becomes the starting address (CSADDR) for the first control section in the input sequence.
- The control section name from Header record is entered into ESTAB, with value given by CSADDR. All external symbols appearing in the Define record for the control section are also entered into ESTAB. Their addresses are obtained by adding the value specified in the Define record to CSADDR.
- When the End record is read, the control section length CSLTH (which was saved from the End record) is added to CSADDR. This calculation gives the starting address for the next control section in sequence.

CSADDR= CSADDR+CSLTH

- At the end of Pass 1, ESTAB contains all external symbols defined in the set of control sections together with the address assigned to each.

Algorithm for Pass 2 of a Linking Loader:

Pass 2:

```
begin
set CSADDR to PROGADDR
set EXECADDR to PROGADDR
while not end of input do
    begin
        read next input record {Header record}
        set CSLTH to control section length
        while record type ≠ 'E' do
            begin
                read next input record
                if record type = 'T' then
                    begin
                        {if object code is in character form, convert
                            into internal representation}
                        move object code from record to location
                            (CSADDR + specified address)
                    end {if 'T'}
                else if record type = 'M' then
                    begin
                        search ESTAB for modifying symbol name
                        if found then
                            add or subtract symbol value at location
                                (CSADDR + specified address)
                        else
                            set error flag (undefined external symbol)
                    end  {if 'M'}
            end {while ≠ 'E'}
        if an address is specified {in End record} then
            set EXECADDR to (CSADDR + specified address)
        add CSLTH to CSADDR
    end  {while not EOF}
jump to location given by EXECADDR {to start execution of loaded program}
end {Pass 2}
```

Explanation for Pass 2 of Linking Loader:

- Pass 2 of our loader performs the actual loading, relocation, and linking of the program.
- As each Text record is read, the object code is moved to the specified address (plus the current value of CSADDR).
- When a Modification record is encountered, the symbol whose value is to be used for modification is looked up in ESTAB. This value is then added to or subtracted from the indicated location in memory.
- The last step performed by the loader is usually the transferring of control to the loaded program to begin execution.
- The End record for each control section may contain the address of the first instruction in that control section to be executed. Our loader takes this as the transfer point to begin execution. If more than one control section specifies a transfer address, the loader arbitrarily uses the last one encountered.
- If no control section contains a transfer address, the loader uses the beginning of the linked program (i.e., PROGADDR) as the transfer point.
- Normally, a transfer address would be placed in the End record for a main program, but not for a subroutine.

To make Pass 1 and Pass 2 efficient:

- The algorithm can be made more efficient. Assign a reference number, which is used (instead of the symbol name) in Modification records, to each external symbol referred to in a control section. we always assign the reference number 01 to the control section name.
- The main advantage of reference number mechanism is that it avoids multiple searches of ESTAB for the same symbol during the loading of a control section.

Symbol and Addresses in PROGA, PROGB and PROGC are as shown below. These are the entries of ESTAB.

PROGA:

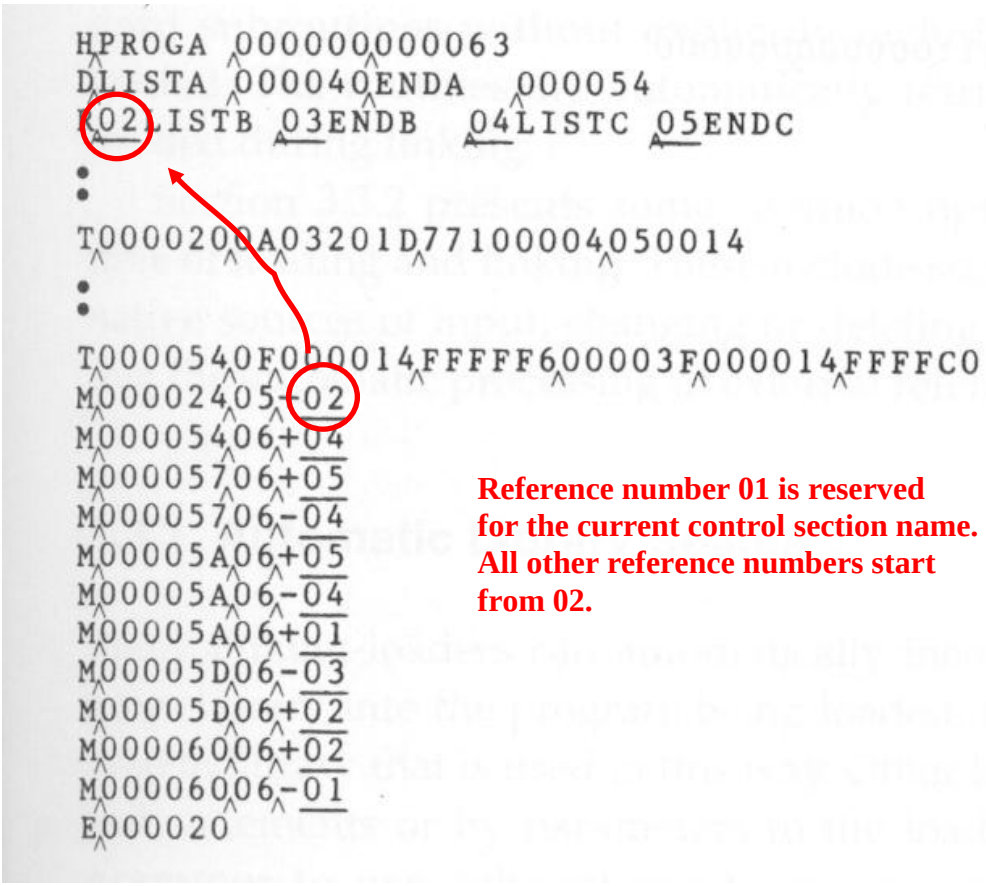
Ref No.	Symbol	Address
1	PROGA	4000
2	LISTB	40C3
3	ENDB	40D3
4	LISTC	4112
5	ENDC	4124

PROGB:

Ref No.	Symbol	Address
1	PROGB	4063
2	LISTA	4040
3	ENDA	4054
4	LISTC	4112
5	ENDC	4124

PROGC:

Ref No.	**Symbol**	**Address**
1	**PROGC**	**40E2**
2	**LISTA**	**4040**
3	**ENDA**	**4054**
4	**LISTB**	**40C3**
5	**ENDB**	**40D3**



```
HPROGA 000000000063
DLISTA 000040ENDA  000054
R02LISTB 03ENDB  04LISTC 05ENDC
.
.
T0000200A03201D771000040500 14
.
.
T0000540F000014FFFFF600003F000014FFFFC0
M00002405+02
M00005406+04
M00005706+05
M00005706-04
M00005A06+05
M00005A06-04
M00005A06+01
M00005D06-03
M00005D06+02
M00006006+02
M00006006-01
E000020
```

Reference number 01 is reserved for the current control section name. All other reference numbers start from 02.

```
HPROGB 00000000007F
DLISTB 000060ENDB  000070
R02LISTA 03ENDA  04LISTC 05ENDC
.
.
T0000360B031000007720270 5100000
.
.
T0000700F000000FFFFF6FFFFFFFFFF0000060
M00003705+02
M00003E05+03
M00003E05-02
M00007006+03
M00007006-02
M00007006+04
M00007306+05
M00007306-04
M00007606+05
M00007606-04
M00007606+02
M00007906+03
M00007906-02
M00007C06+01
M00007C06-02
E
```

```
HPROGC 000000000051
DLISTC 000030ENDC  000042
R02LISTA 03ENDA  04LISTB 05ENDB
.
.
T0000180C031000007710000405100000
.
.
T0000420F000030000008000011000000000000
M00001905+02
M00001D05+04
M00002105+03
M00002105-02
M00004206+03
M00004206-02
M00004206+01
M00004806+02
M00004B06+03
M00004B06-02
M00004B06-05
M00004B06+04
M00004E06+04
M00004E06-02
E
```

Machine-Independent Loader Features:

Loading and linking are often thought of as OS service functions. Therefore, most loaders include fewer different features than are found in a typical assembler. They include the use of an automatic library search process for handling external reference and some common options that can be selected at the time of loading and linking.

Automatic Library Search:

- Many linking loaders can automatically incorporate routines from a subprogram library into the program being loaded. (E.g., the standard C library)
- The subroutines called by the program are automatically fetched from the library, linked with the main program, and loaded.
- The programmer does not need to take any action beyond mentioning the subroutine names as external references in the source program.
- Linking loader that support automatic library search must keep track of external symbols that are referred to, but not defined, in the primary input to the loader.
- At the end of pass 1, the symbols in ESTAB that remain undefined represent unresolved external references.
- The loader searches the library for routines that contain the definitions of these symbols, and processes the subroutines found by this search process exactly as if they had been part of the primary input stream.
- The subroutines fetched from a library in this way may themselves contain external references. It is necessary to repeat the library search process until all references are resolved.
- If unresolved references remain after the library search is completed, they are treated as errors.
- If a symbol (or a subroutine name) is defined both in the source program and in the library, the one in the source program is used first.
- A programmer can make his own library easily on UNIX by using the "ar" command.

Loader Options:

- Allow the user to specify options that modify the standard processing
- Specified using a command language
- Specified as a part of job control language that is processed by the operating system
- Specified using loader control statements in the source program.

Five options are:

1. **Include** program-name (library name)
   Direct the loader to read the designated object program from a library
2. **Delete** csect-name
   Instruct the loader to delete the named control sections from the set of programs being loaded

3. **Change** name1, name2
   Cause the external symbol name1 to be changed to name2 wherever it appears in the program
4. **LIBRARY** MYLIB – search MYLIB library before standard libraries
5. **NOCALL** STDDEV, PLOT, CORREL – no loading and linking of unneeded routines.

Example:

- In the COPY program, we write two subroutines RDREC and WRREC to perform read records and write records.
- Suppose that the computer system provides READ and WRITE subroutines which have similar but advanced functions.
- Without modifying the source program and reassembling it, we can use the following loader options to make the COPY object program use READ rather than RDREC and WRITE rather than WRREC.

```
LIBRARY UTLIB
INCLUDE READ (UTLIB)
INCLUDE WRITE (UTLIB)
DELETE RDREC, WRREC
CHANGE RDREC, READ
CHANGE WRREC, WRITE
NOCALL SQRT, PLOT
```

The commands are, use UTLIB (say utility library), include READ and WRITE control sections from the library, delete the control sections RDREC and WRREC from the load, the change command causes all external references to the symbol RDREC to be changed to the symbol READ, similarly references to WRREC is changed to WRITE, Finally, no call to the functions SQRT, PLOT, if they are used in the program.

Loader Design Options:

There are some common alternatives for organizing the loading functions, including relocation and linking.

Linking Loaders: Perform all linking and relocation at load time.
Linkage Editors: Which perform linking prior to load time.
Dynamic Linking: Linking function is performed at execution time.
Bootstrap Loaders: used to run stand-alone programs independent of the operating system or the system loader.

Linkage Editors:

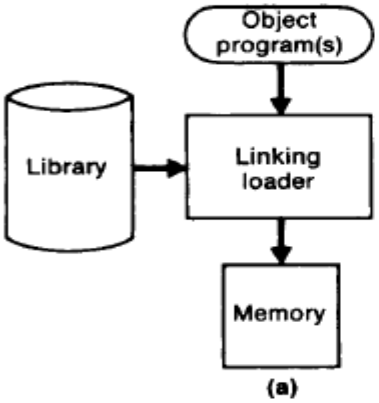
- A linkage editor produces a linked version of the program (called load module or an executable image), which is written to a file or library for later execution.
- When the user is ready to run the linked program, a simple relocating loader can be used to load the program into memory.
- The only object code modification necessary is the addition of an actual load address to relative values within the program.

- The linkage editor performs relocation of all control sections relative to the start of the linked program. Thus all items that need to be modified at load time have values that are relative to the start of the linked program.
- This means that the loading can be accomplished in one pass with no external symbol required. This involves less overhead than using a linking loader.
- If a program is to be executed many times without being reassembled, the use of a linkage editors, reduces the overhead required, Resolution of external references and library searching are only performed once.
- A linking loader searched libraries and resolves external references every time the program is executed.
- Even though linking is performed prior to load time in the case of linkage editor information's about external references is retained in the linked program. This makes relinking, modifying external references etc…,
- If this information is not retained, the linked program can only be loaded and executed and it cannot be reprocessed by the linkage editor.
- Linkage editors can perform many useful functions besides simply preparing an object program for execution.

- **Advantages**:
  One linking and the most of relocation for many executions.
  Possibility to create subroutines libraries.
  Actual loading is very simple.

- **Disadvantages**:
  Recompiling for nearly every execution (program development).
  Infrequently used programs (storage waste).

Comparison between Linkage Editor and Linking Loader



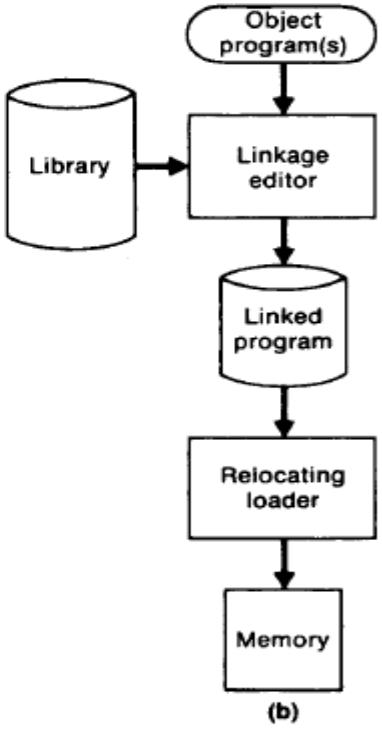


Fig (a): Linking Loader **Fig (b): Linkage Editor**

Linking Loader	**Linkage Editors**
Performs linking and relocation at load time.	Linking is done prior to load time
Loads the linked program directly into the memory. Therefore steps for writing and reading the linked program are avoided.	Writes a linked version of program, into a file which is later executed by the relocation loader.
Linking loader searches libraries and resolves external references every time the program is executed	Resolution of external references and library searching are performed only once in linkage editor.
Used in situations where program has to reassembled for every execution Eg: program development and testing	Linkage editor can be efficiently used when a program is to be executed many times without being reassembled, each time
Linking loader has less flexibility and control	Linkage editors offer more flexibility and control
Less complexity	Increase in complexity
The Loading may requires 2 Passes	The Loading can be accomplished in one Pass

Consider for example,

A program PLANNER that uses a large number of subroutines. Suppose that one subroutine name PROJECT used by the program is changed. After the new

version of PROJECT is reassembled the linkage editor can be used to replace this subroutine.

The following is a typical sequence of linkage editor commends which is used to accomplish this task.

```
INCLUDE PLANNER (PROGLIB)
DELETE PROJECT                {Delete from exiting PLANNER}
INCLUDE PROJECT (NEWLIB)      {Include the new version}
REPLACE PLANNER (PROGLIB)
```

Linkage editors can also be used to build packages of subroutine or other control sections that are generally used together, like to read and write data blocks, to encode and decode the data blocks etc., In linkage editor could be combine the appropriate subroutines into a package with a command sequence like the following,

```
INCLUDE READ (UTLIB)
INCLUDE WRITE (UTLIB)
INCLUDE ENCODE (UTLIB)
INCLUDE DECODE (UTLIB)
INCLUDE BLOCK (UTLIB)
INCLUDE DEBLOCK (UTLIB)
SAVE LNKMOD (SUBLIB)
```

The linked module LNKMOD could be indexed in the directory of SUBLIB. Search of SUBLIB would retrieve LNKMOD, which has all the references between the READ, WRITE… resolved, so linkage need not reprocessed when each time the program is linked.
Linkage editors often allow the user to specify that external references are not to be resolved automatic library search. In such case only the external references between user written routines would be resolved. Linking loader is used to combine linked user subroutine with LNKMOD at execution time.

Dynamic Linking:

- The scheme that postpones the linking functions until execution.
- A subroutine is loaded and linked to the rest of the program when it is first called. This type of function is called dynamic linking, dynamic loading or load on call.

Advantages:

- Dynamic linking is often used to allow several executing programs to share one copy of a subroutine or library.
- Dynamic linking also makes it possible for one object to be shared by several programs.
- Dynamic linking provides the ability to load the routines involved are large or have many external references; this can result in substantial savings of time and memory space.
- It avoids the necessity of loading the entire library for each execution.
- There are number of different mechanism that can be used to accomplish the actual loading and linking of a called subroutine.

In this method which routines that are to be dynamically loaded must be called via on operating system service request. This method could also be thought of as a request to a part of the loader that is kept in memory during execution of the program.

The Loading and calling of a subroutine using Dynamic Linking is shown in below Figures

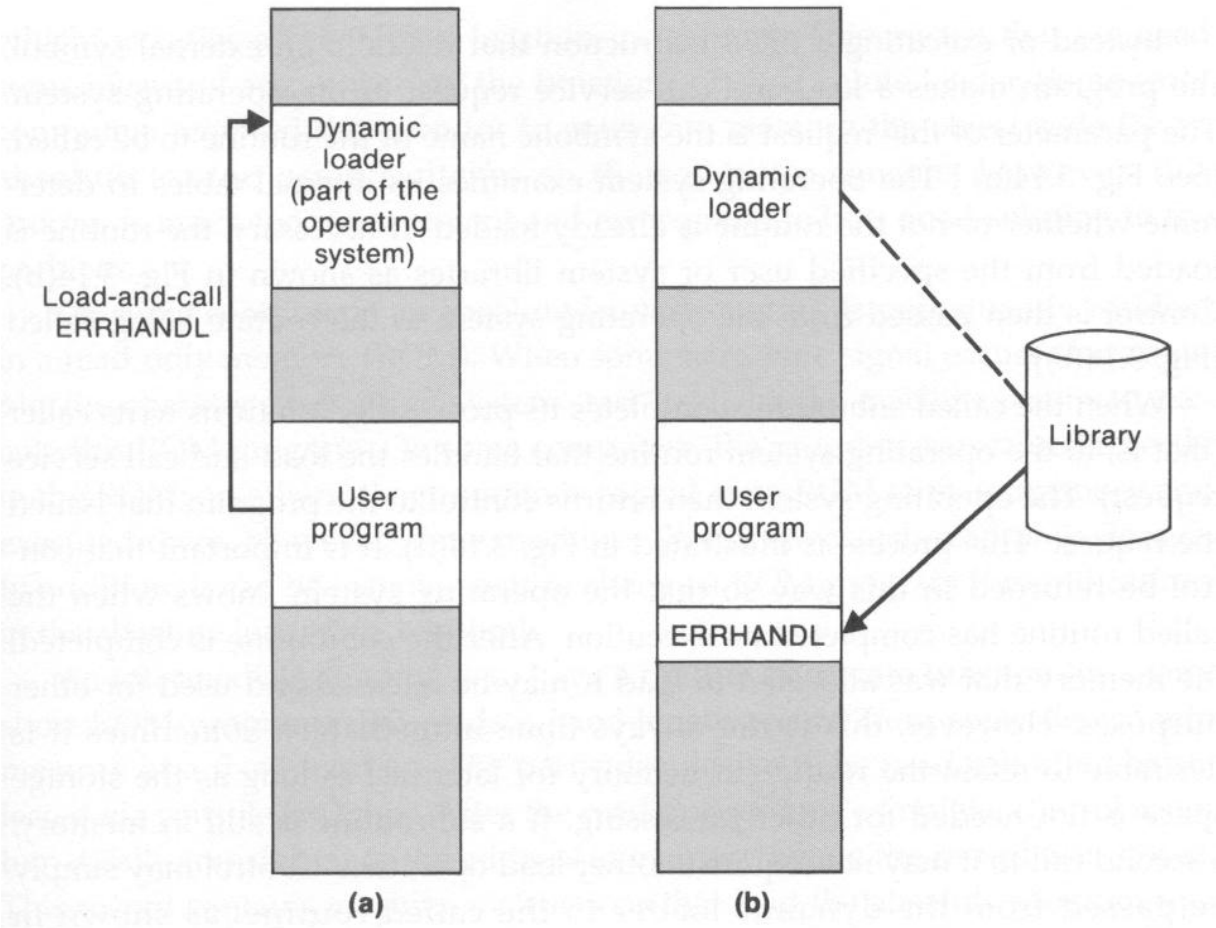


Fig (a): Instead of executing a JSUB instruction that refers to an external symbol the program makes a load and call service request to the operating system. The parameter of this request is the symbolic name of the routine to be called.

Fig (b): The operating system examines its internal tables to determine whether the routine is already loaded or not. If necessary, the routine is loaded from this specified user or system libraries.

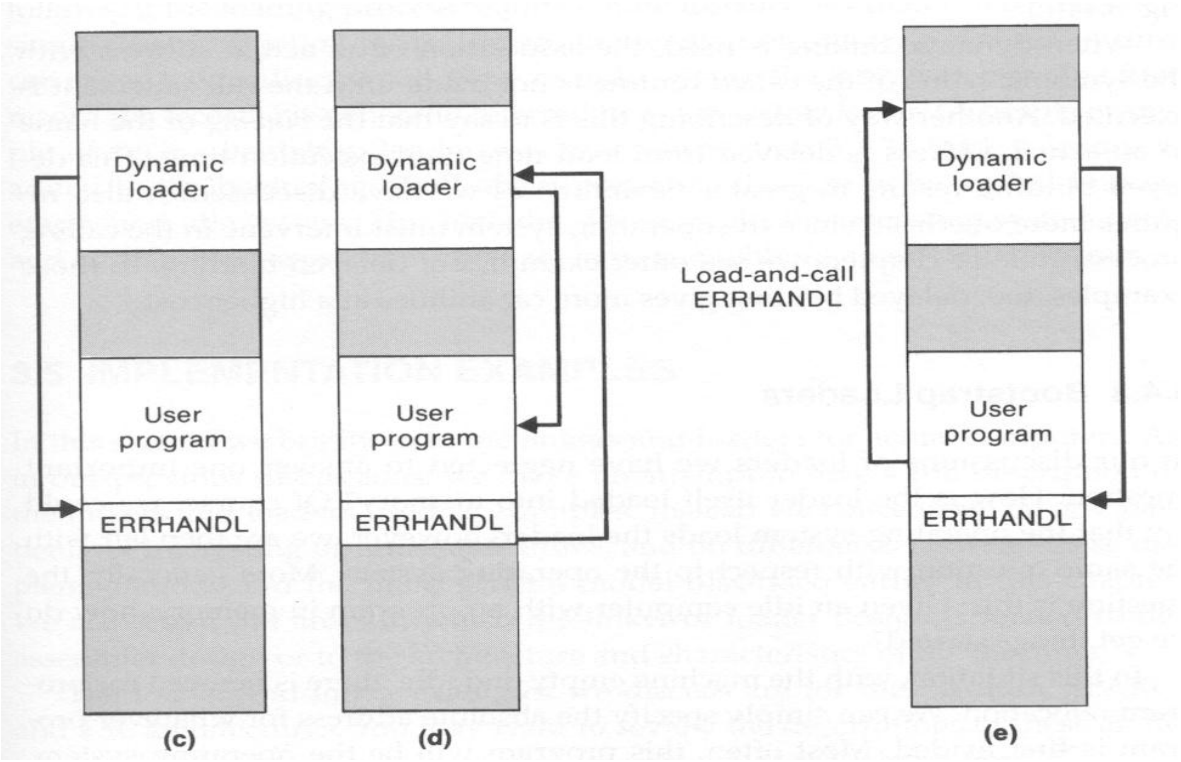


Fig (c): Control is then passed from the operating system to the routing being called.

Fig (d): When the called subroutine completes it processing it returns to its caller. The operating system then returns control to the program that issued the request. The control returned in this way so that the operating system knows when the called subroutine has completed its execution.

Fig (e): If a subroutine is still in memory, a second call to it may not require another load operation. Control may be simply be passed from the dynamic loader to the called routine. When dynamic linking is used, the binding of the name to an actual address is delayed from load time until execution time. This delayed binding gives greater flexibility.

Bootstrap Loaders:

- Given an idle computer with no program in memory, how do we get things started?

- On some computers, an absolute loader program is permanently resident in a read-only memory (ROM). When some hardware signal occurs, the machine begins to execute this ROM program. This is referred to as a bootstrap loader.
- Reads a fixed-length record form some device into memory at a fixed location.
- After the read operation is complete, control is automatically transferred to the address in memory.
- If the loading process requires more instructions than can be read in a single record, this first record causes the reading of others, and these in turn can cause the reading of more records.

Implementation Examples:

Contains brief description of loaders and linkers for actual computers. They are, MS-DOS Linker - Pentium architecture, SunOS Linkers - SPARC architecture, and, Cray MPP Linkers – T3E architecture.

MS-DOS Linker:

- It is used for Pentium and X86 systems.
- MS DOS compiler and assemblers produce object modules. Object module has the file extension as **.OBJ**
- Each object module describes structure of program. It also contain binary image of translated instructions and program data.
- It combines one or more object modules to form a complete executable program with extension as **.EXE**

The following table illustrates the typical MS-DOS object module

Record Types	**Description**
THEADR	**Translator header**
TYPDEF **PUBDEF** **EXTDEF**	**External symbols and references**
LNAMES **SEGDEF** **GRPDEF**	**Segment definition and grouping**
LEDATA **LIDATA**	**Translated instructions and data**
FIXUPP	**Relocation and linking information**
MODEND	**End of object module**

The below figure gives the function and meaning of above record types:

Record Types	**Meaning**	**Function**
THEADR	Translator Header Record	Record specifies the name of the object module.
MODEND	Module End	Record marks end of the module.
PUBDEF	Public Names	Contains list of all the public names used

	Definition Record	in segment.
EXTDEF	External Names Definition Record	Contains a list of all external references made in the program.
TYPDEF	Type Definition Record	Specifies data types of symbols.
SEGDEF	Segment Definition Record	Describe segments in the object module, including their name, length and alignment.
LNAMES	List of Names Record	Contains a list of all the segment and class names used in the program.
GRPDEF	Group Definition Record	Specifies how these segments are combined into group.
LEDATA	Logical Enumerated Data Record	Contains translated instruction and data from source program.
LIDATA	Logical Iterated Data Record	Specify translated instruction and data that occur in a repeating pattern.
FIXUPP	Fix Up Record	Carry out address modifications that are associated with relocation and grouping of segments within the program.

Process of MS-DOS link

Perform two passes namely pass1 and pass2.

During PASS1:

1. Starting address of each segment is computed
2. Place segments in executable program in same order.
3. The symbol table is constructed in which an address is associated with segment and external symbol
4. If there are unresolved external symbols, LINK searches specific library.

During PASS2:

1. Extract translated instructions and data from object modules.
2. Build image of executable program in memory.
3. Memory less uses temporary disk file.
4. Pass 2 of link process LEDDATA and LIDDATA record with the FIXUPP record.
5. Place binary data from LEDDATA and LIDDATA records into memory image. Relocations performed and external references are resolved.
6. Relocation that involve starting address of the segment are added to the table of segment FIXUPPS.
7. After memory image is complete, LINK writes it to executable file (.EXE).
8. This File includes a header that contains:
   - Fix up segment table.
   - Memory requirement information.
   - Initial content for CS (Counter Segment) and SP (Segment Pointer) registers.